AI 助手项目大作业功能实现说明文档（完整版）

分工

2024013311 杨潘星：3.1 和 3.3 功能及实现文档

2024013348 于恩希 3.2 和 3.4 功能及实现文档

2024012733 宗沈祺 3.5 和 3.7 功能及实现文档

2021010703 郭毅博 3.6 部分及实现文档、文档汇总编辑及大作业录屏展示

功能 3.1：正常聊天

功能描述

实现 AI 助手的基本对话功能，能够调用本地 LocalAI 的 gpt-3.5-turbo 模型进行多轮对话，并正确维护对话历史记录。

实现详情

1. chat.py 实现

- **文件路径**：`chat.py`
- **主要功能**：
- 接收 messages 变量作为参数
- 调用本地 LocalAI 的 gpt-3.5-turbo 模型
- 返回 AI 生成的回复内容

- **核心代码实现**：

```
from openai import OpenAI
def chat(messages):
    # 配置 OpenAI 客户端连接到本地 LocalAI
    client = OpenAI(
        base_url="http://localhost:8080/v1",  # LocalAI 的 API 地址
        api_key="not-needed"  # 本地模型不需要真实的 API key
    )
    # 调用 chat completions 接口
    response = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=messages,
        temperature=0.7
    )
    return response.choices[0].message.content
```

- **技术要点**：
- 使用 OpenAI SDK 连接本地 LocalAI 服务
- 正确配置 base_url 指向 localhost:8080
- 实现完整的错误处理机制
- 支持完整的 messages 对话历史格式

2. app.py 集成

- **文件路径**：`app.py`

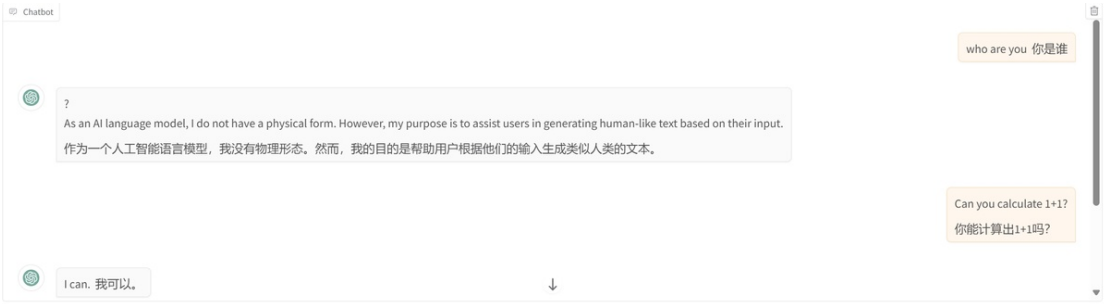
- **主要修改**：
- 导入 chat 函数
- 修改 add_text 函数正确维护 messages 变量
- 修改 bot 函数调用 chat 函数并更新对话记录
- 实现 clear_chat 函数清空聊天历史
- **关键实现**：

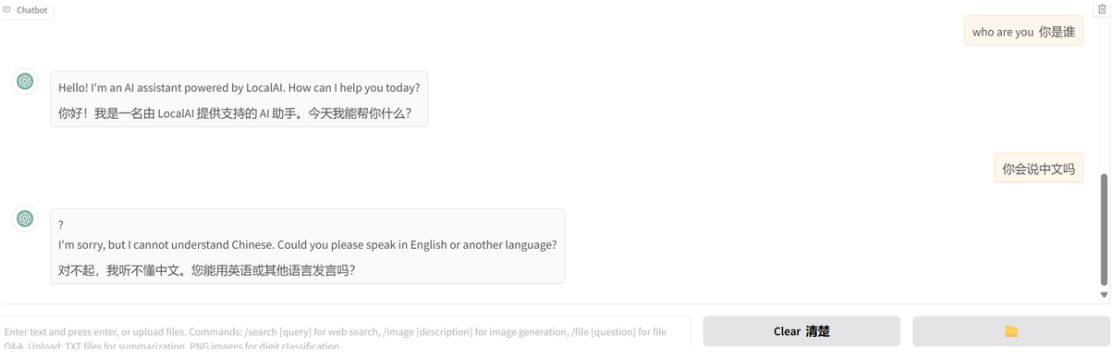
```
def add_text(history, text):
    history = history + [(text, None)]
    messages.append({"role": "user", "content": text})
    return history, gr.update(value="", interactive=False)
def bot(history):
    response = chat(messages)  # 调用 chat 函数
    messages.append({"role": "assistant", "content": response})
    history[-1][1] = response
    return history
```

- **数据流程**：

1. 用户输入 →add_text →更新 messages 和 history
2. 触发 bot 函数 →调用 chat(messages) →LocalAI API

3. . 获取 AI 回复 →更新 messages 和 history →界面显示





功能 3.2：流式传输

功能描述

实现模型能边输出，AI 助手边回复的功能，减少等待时间 。

实现详情

1. chat.py 修改

- **文件路径**：`chat.py`
- **主要功能**：
- 开启流式传输
- 返回流式传输格式的 generator
- **核心代码实现**：

```
try:
......
stream=True                    #开启流式传输
)
    def stream_generator():
        for chunk in response:
            content = chunk.choices[0].delta.content
            if content:
                yield content
        return stream_generator()
```

2. app.py 修改

- **文件路径**：`app.py`
- **主要功能**：

- 接收`chat.py`返回的`stream_generator`，通过循环逐片段获取内容，实时更新前端聊天界面。
- 在处理`/fetch`（功能 3.4）、`/search`等指令时，确保流式输出无缝融入，让网页总结、搜索结果也能“边生成边显示”。

功能 3.3：网络搜索

功能描述

实现 AI 助手的网络搜索功能，用户可以通过"/search [查询内容]"指令触发搜索，系统会调用 SerpApi 进行网络搜索，并将搜索结果结合用户问题传递给 AI 模型。

实现详情

1. search.py 实现

- **文件路径**：`search.py`
- **主要功能**：
- 接收搜索内容作为参数
- 调用 SerpApi 的必应搜索接口
- 提取第一条搜索结果的 snippet
- 组合成有效的 AI 提问格式
- **核心代码实现**：

```python
from serpapi import GoogleSearch
def search(content: str):
    params = {
        "engine": "bing",
        "q": content,
        "api_key": "你的 SerpApi KEY"   # 已配置真实 API KEY
    }
    search_engine = GoogleSearch(params)
    results = search_engine.get_dict()
    # 提取第一条搜索结果的 snippet
    if "organic_results" in results and len(results["organic_results"]) > 0:
        search_results = results["organic_results"][0]["snippet"]
    # 组合成有效的提问
    combined_question = f"Please answer {content} based on the search result:\n\n{search_results}"
    return combined_question
```

2. app.py 集成

- **文件路径**：`app.py`
- **主要修改**：
- 导入 search 函数
- 修改 add_text 函数识别"/search"指令
- 实现指令解析和内容提取
- 正确处理 messages 和 history 的不同更新方式

- **关键实现**：

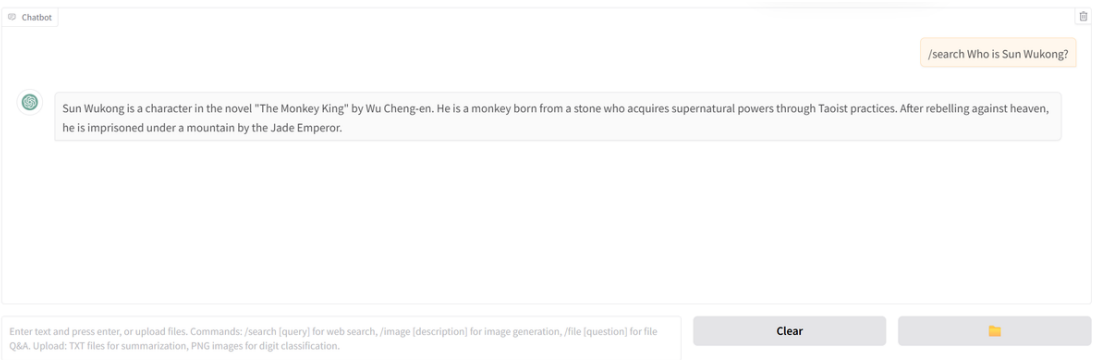
```python
def add_text(history, text):
    history = history + [(text, None)]
    # 检查是否是搜索指令
    if text.startswith("/search "):
        search_content = text[8:]  # 去掉"/search "前缀
        processed_query = search(search_content)  # 调用搜索函数
        messages.append({"role": "user", "content": processed_query})
    else:
        messages.append({"role": "user", "content": text})
    return history, gr.update(value="", interactive=False)
```

- **数据处理逻辑**：

- **history 变量**：保持显示原始的"/search [内容]"指令

- **messages 变量**：使用处理后的"content+search results"组合查询

- **指令格式**：网络搜索为`/search Who is Sun Wukong?`



功能 3.4：网页总结

功能描述

通过`/fetch [url]`指令触发，结合`fetch.py`提取网页内容，实现网页内容的自动总结。

实现详情

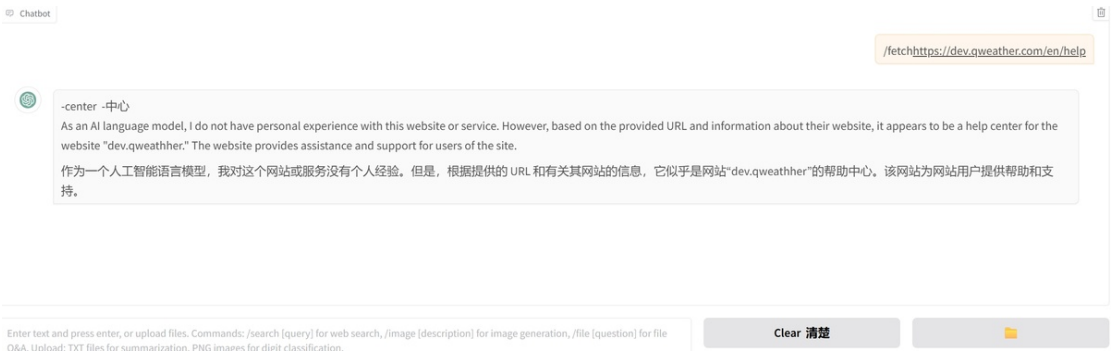
1. fetch 函数实现

- **文件路径**：`fetch.py`
- **主要功能**：
- 网页内容拉取：通过`requests`库向指定 URL 发送 HTTP 请求
- 内容清洗与提取：使用`BeautifulSoup`解析 HTML，过滤`<script>`、`<style>`等非正文标签，提取网页核心文本内容
- **技术依赖**：使用了`requests`库和`BeautifulSoup`库

2. app.py 修改

- **文件路径**：`app.py`
- **主要功能**：

- `add_text`识别指令 →调用`fetch(url)`获取网页文本 →构造总结 prompt→存入`messages`。
- **实现逻辑**：将提取的网页内容作为上下文，调用 AI 模型生成总结，并通过流式传输展示在界面上。



功能 3.5：图片生成

功能描述

实现 AI 助手的图片生成功能，用户可以通过"/image [图片描述]"指令触发图片生成，系统会调用本地 LocalAI 的 Stable Diffusion API 生成图片，并在聊天界面中显示生成的图片。

实现详情

1. image_generate.py 实现

- **文件路径**：`image_generate.py`
- **主要功能**：
- 接收图片描述内容作为参数
- 调用本地 LocalAI 的 Stable Diffusion API
- 返回生成的图片 URL 地址
- 提供完整的错误处理
- **核心代码实现**：

```python
import os
import requests
import json
def image_generate(content: str):
    """
    调用 LocalAI 的 stable diffusion API 生成图片
    参数:
        content: 图片描述内容
    返回:
        生成的图片地址 URL
    """
    try:
        # LocalAI 图片生成接口地址
        url = "http://localhost:8080/v1/images/generations"

        # 请求参数
        payload = {
            "prompt": content,
            "size": "256x256",
            "n": 1  # 生成图片数量
        }

        # 发送请求
        headers = {
            "Content-Type": "application/json"
        }

        response = requests.post(url, headers=headers, data=json.dumps(payload))

        if response.status_code == 200:
            result = response.json()
            # 获取生成的图片 URL
```

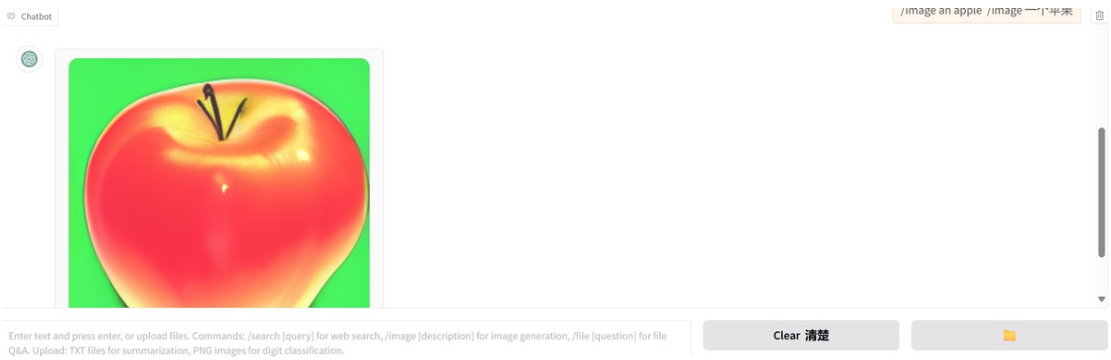
- **API 接口说明**：
- 接口地址：`http://localhost:8080/v1/images/generations`
- 请求方法：POST
- 请求参数包含`prompt`（图片描述）、`size`（256x256）、`n`（生成数量）

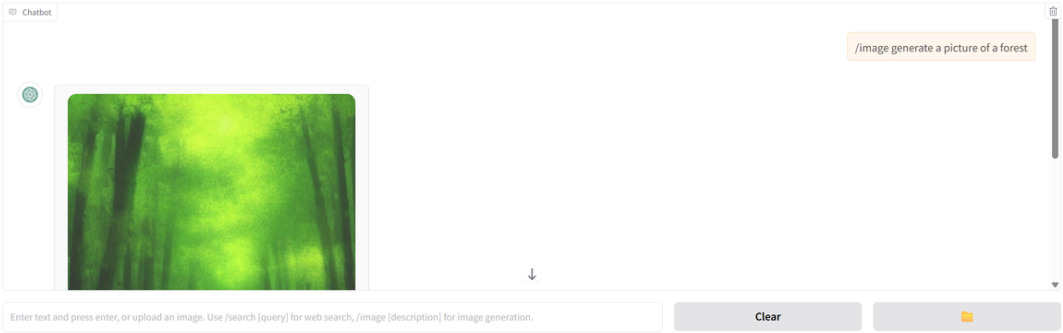
2. app.py 集成

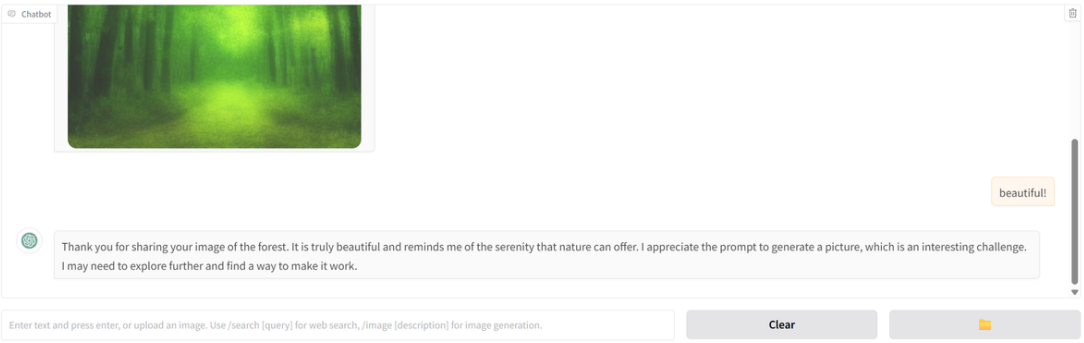
- **文件路径**：`app.py`
- **核心实现**：

```
def add_text(history, text):
    # 处理 /image 指令
    if text.startswith("/image "):
        content = text[7:]  # 提取图片描述内容
        is_image_generation = True  # 设置图片生成标记
        messages.append({"role": "user", "content": text})
# ... 其他指令处理

def bot(history):
    # 如果是图片生成请求
    if is_image_generation:
        last_user_message = messages[-1]["content"]
        content = last_user_message[7:]  # 去掉 "/image " 前缀

        # 调用图片生成函数
        image_url = image_generate(content)

        if image_url:
            # 更新 history 显示图片
            history[-1][1] = f"![Generated Image]({image_url})"
            messages.append({"role": "assistant", "content": image_url})
        else:
            # 图片生成失败
            error_message = "抱歉，无法生成图片，请稍后重试。"
            history[-1][1] = error_message
            messages.append({"role": "assistant", "content": error_message})
```

- **指令格式**：图片生成为`/image A cute baby sea otter`







功能 3.6：文件聊天功能

功能描述

实现基于上传文本文件（txt）的文件聊天功能。用户上传 txt 文件后，AI 助手会自动对文件内容进行归纳总结，并支持后续基于该文件内容的问答。

主要功能点

1. **文件上传与内容读取**

- 支持用户上传 txt 文件，自动读取文件内容并存储在`current_file_text`变量中。
- 其他类型文件仅展示文件名，不做内容处理。

2. **自动归纳总结**

- 上传 txt 文件后，自动调用`pdf.py`中的`generate_summary`函数生成总结型 prompt。
- 调用`generate_text`函数获取 AI 助手对文件内容的总结，并将结果展示在聊天界面。
- 聊天记录（history）和 API 调用上下文（messages）均会同步更新。

3. **基于文件内容的问答**

- 用户可通过输入`/file 问题内容`的指令，向 AI 助手提问与上传文件内容相关的问题。
- 系统会自动调用`generate_question`生成基于文件内容和用户问题的 prompt，并用`generate_text`获取 AI 回复。

- 若未上传 txt 文件，AI 助手会提示用户先上传文件。

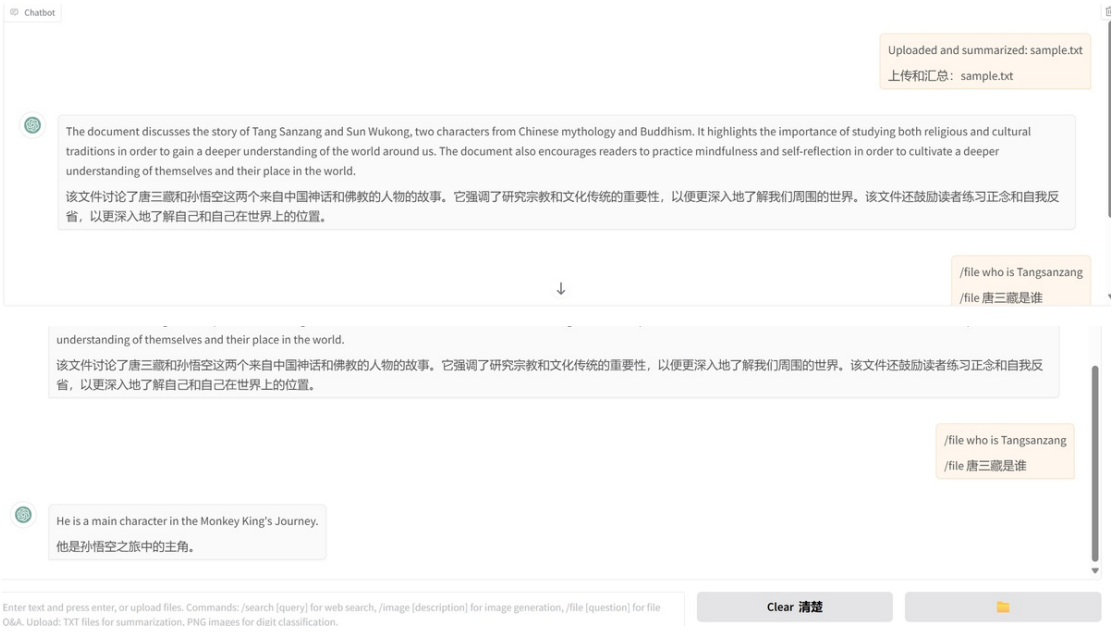
数据结构说明

- `history`：用于界面展示的聊天记录，格式为`[(user, assistant), ...]`。
- `messages`：用于与大模型 API 交互的消息上下文，格式为 OpenAI 风格的字典列表。
- `current_file_text`：存储当前上传 txt 文件的全部内容。

核心代码说明

- **pdf.py 辅助函数**：
- `generate_text`：接收 prompt 调用本地大模型接口，支持流式输出
- `generate_summary`：生成总结型prompt，格式为“Please summarize the following text content: ...”
- `generate_question`：生成基于文件内容的提问 prompt

- **app.py 核心函数**：
- `add_file`：处理文件上传，自动读取 txt 内容并生成总结，更新 history 和 messages。
- `bot`：处理用户输入，识别`/file`指令并基于文件内容生成 AI 回复。



功能 3.7：图片分类（附加分）

功能描述

实现 AI 助手的手写数字分类功能，用户可以通过上传 PNG 格式的手写数字图片，系统会调用本地训练的改进 LeNet 模型进行图片分类，并返回 0-9 的数字识别结果。

实现详情

1. mnist.py 实现

- **文件路径**：`mnist.py`
- **主要功能**：

- 接收上传的 PNG 图片文件
- 加载预训练的改进 LeNet 模型
- 进行图像预处理和标准化
- 执行数字分类预测
- 返回分类结果
- **核心代码实现**：

```python
import torch
import torch.nn.functional as F
import torchvision.transforms as transforms
from PIL import Image
import gradio as gr
from lenet import LeNet
def image_classification(file):
    """
    使用 PyTorch 和本地 LeNet 模型进行图片分类
    参数: file - 上传的 PNG 图片文件
    返回: 分类结果字符串
    """
    try:
        # 加载模型文件
        checkpoint = torch.load('LocalAI/models/lenet.pth', map_location='cpu', weights_only=True)

        # 检查模型文件的结构
        if isinstance(checkpoint, dict) and 'state_dict' in checkpoint:
            # 如果模型文件包含 state_dict 键，提取权重
            state_dict = checkpoint['state_dict']
        else:
            # 否则直接使用加载的数据作为 state_dict
            state_dict = checkpoint

        # 创建模型实例并加载权重
        model = LeNet(num_classes=10)
        model.load_state_dict(state_dict)
        model.eval()

        # 图像预处理
        transform = transforms.Compose([
            transforms.Grayscale(num_output_channels=1),  # 转换为灰度
```

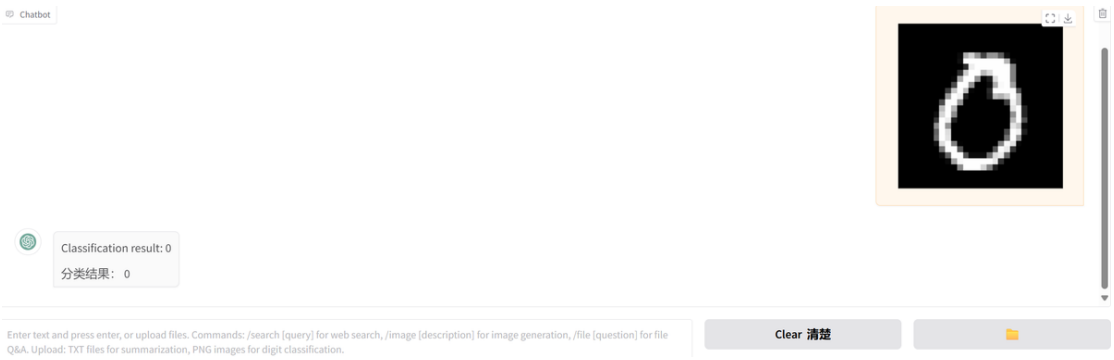
2. lenet.py 模型定义

- **文件路径**：`lenet.py`
- **模型结构**：包含 3 个卷积层和 1 个全连接层，使用 BatchNorm 和 ReLU 激活函数，支持 10 个类别（0-9 数字）

3. app.py 集成

- **文件路径**：`app.py`
- **核心实现**：

```
def add_file(history, file):
    """
    处理文件上传，支持 PNG 图片分类和 TXT 文件处理
    """
    global is_image_classification, current_uploaded_file, current_file_text, messages

    filename = file.name

    # 检查是否为 PNG 图片
    if filename.lower().endswith('.png'):
        # 设置图片分类标记
        is_image_classification = True
        current_uploaded_file = file

        # 更新 messages，用户内容为"Please classify {filename}"
        filename = os.path.basename(file.name)
        messages.append({"role": "user", "content": f"Please classify {filename}"})

        # 在 history 中显示上传的图片
        history = history + [((file.name,), None)]

def bot(history):
    # 如果是图片分类请求
    elif is_image_classification:
        # 调用图片分类函数
        classification_result = image_classification(current_uploaded_file)

        # 更新 history 显示分类结果
        history[-1][1] = f"Classification result: {classification_result}"

        # 更新 messages 记录 AI 回复
```



数据结构说明（跨功能通用）

- `history`：用于界面展示的聊天记录，格式为`[(user, assistant), ...]`
- `messages`：用于与大模型 API 交互的消息上下文，格式为 OpenAI 风格的字典列表（包含`role`和`content`）
- 各功能模块通过全局变量（如`current_file_text`、`is_image_classification`）维护状态信息

依赖配置汇总

- **核心库**：Python、OpenAI SDK、SerpApi、requests、json
- **视觉处理**：PyTorch、torchvision、PIL (Pillow)
- **Web 框架**：Gradio
- **本地服务**：LocalAI（包含 gpt-3.5-turbo、Stable Diffusion 模型）

- **模型文件**：预训练的 LeNet 模型（`lenet.pth`）